**INTELIGENCIA ARTIFICIAL**
**CON MACHINE LEARNING EN JAVA**

# Proyecto ID3

## Actividad 2 · Árboles de decisión

| Datos del estudiante | Información |
|---|---|
| Nombre | Rodrigo Valdespino Vertiz |
| ID | 00520621 |
| Carrera | Ingeniería en Tecnologías de la Información y Negocios Digitales |
| Campus | Querétaro |
| Periodo académico | Séptimo semestre |

Implementación en Java, cálculo de entropía y validación con los archivos proporcionados para la actividad.

Contenido: descripción de clases, fundamento del algoritmo, resultados reproducibles, instrucciones de ejecución y guía de evidencias.

# 1. Propósito y estructura

El objetivo es construir un árbol de decisión mediante ID3: seleccionar en cada nodo el atributo que aporta la mayor ganancia de información respecto de la clase objetivo. El programa acepta archivos CSV categóricos, muestra sus cálculos y permite ejecutar los cuatro archivos de prueba proporcionados.

## Clases utilizadas

| Clase | Propósito |
|---|---|
| Main | Punto de entrada. Recibe argumentos, coordina el entrenamiento e imprime cálculos, árbol, predicciones y errores. |
| CsvReader | Lee CSV en UTF-8, detecta el separador e interpreta comillas, BOM y saltos de línea. |
| Dataset | Valida encabezados y registros; conserva atributos y columna objetivo en una tabla inmutable. |
| Entropy | Calcula frecuencias, entropía de Shannon y clase mayoritaria. |
| Id3Trainer | Calcula las ganancias de información, selecciona atributos y construye el árbol recursivamente. |
| DecisionNode | Representa decisiones y hojas; clasifica registros nuevos y genera el árbol en texto. |
| Id3Test | Ejecuta 39 pruebas de cálculos, árboles, archivos originales, casos límite y errores de entrada. |

## Organización del proyecto

```
src/id3/              Seis clases de la aplicación
test/id3/             Suite de pruebas
data/proporcionados/  Cuatro CSV originales
docs/evidencias/      Resultados reales de consola
ID3.jar               Aplicación compilada
run.sh / run.bat      Compilación y ejecución
```

Se utiliza Java 17 o superior, sin dependencias externas de ejecución. El proyecto incluye los archivos de importación para Eclipse y codificación UTF-8. La lógica de aprendizaje está implementada desde cero.

# 2. Fundamento y cálculo

## Entropía y ganancia de información

La entropía mide la incertidumbre de las clases. Es cero cuando todos los registros de un subconjunto pertenecen a la misma clase. Se emplean logaritmos de base dos, por lo que el resultado se expresa en bits.

```
H(S) = - sum[p(c) * log2(p(c))]
H(Y | A) = sum[(|Sv| / |S|) * H(Sv)]
IG(S, A) = H(S) - H(Y | A)
```

S es el conjunto de registros, c una clase, A un atributo y Sv el subconjunto con un valor v de ese atributo. La consola muestra H(A), H(Sv), los pesos, la entropía condicional H(Y | A) y la ganancia. ID3 elige por ganancia sobre la clase; H(A) por sí sola no decide la división.

## Ejemplo de los archivos data.csv y Play.csv

Los 14 registros contienen nueve clases Yes y cinco No. Por tanto: H(S) = −(9/14) log2(9/14) − (5/14) log2(5/14) = 0.940286 bits.

Outlook divide los datos en Sunny (2 Yes, 3 No), Overcast (4 Yes, 0 No) y Rain (3 Yes, 2 No). Sus entropías son 0.970951, 0 y 0.970951. La suma ponderada es (5/14)(0.970951) + (4/14)(0) + (5/14)(0.970951) = 0.693536 bits.

| Atributo | H(Play \| A) | Ganancia IG |
|---|---|---|
| Outlook | 0.693536 | 0.246750 |
| Temperature | 0.911063 | 0.029223 |
| Humidity | 0.788450 | 0.151836 |
| Wind | 0.892159 | 0.048127 |

Outlook obtiene la mayor ganancia: 0.940286 − 0.693536 = 0.246750 bits. Se elige como raíz y el cálculo se repite dentro de cada rama con los atributos restantes.

## Reglas de terminación y desempate

Un subconjunto puro produce una hoja. Si no quedan atributos, se devuelve la clase mayoritaria. Los empates de ganancia dentro de $10^{-12}$ respetan el orden del encabezado; los de mayoría, la primera clase encontrada. No se detiene el árbol únicamente porque la ganancia sea cero: las combinaciones posteriores pueden separar casos como XOR.

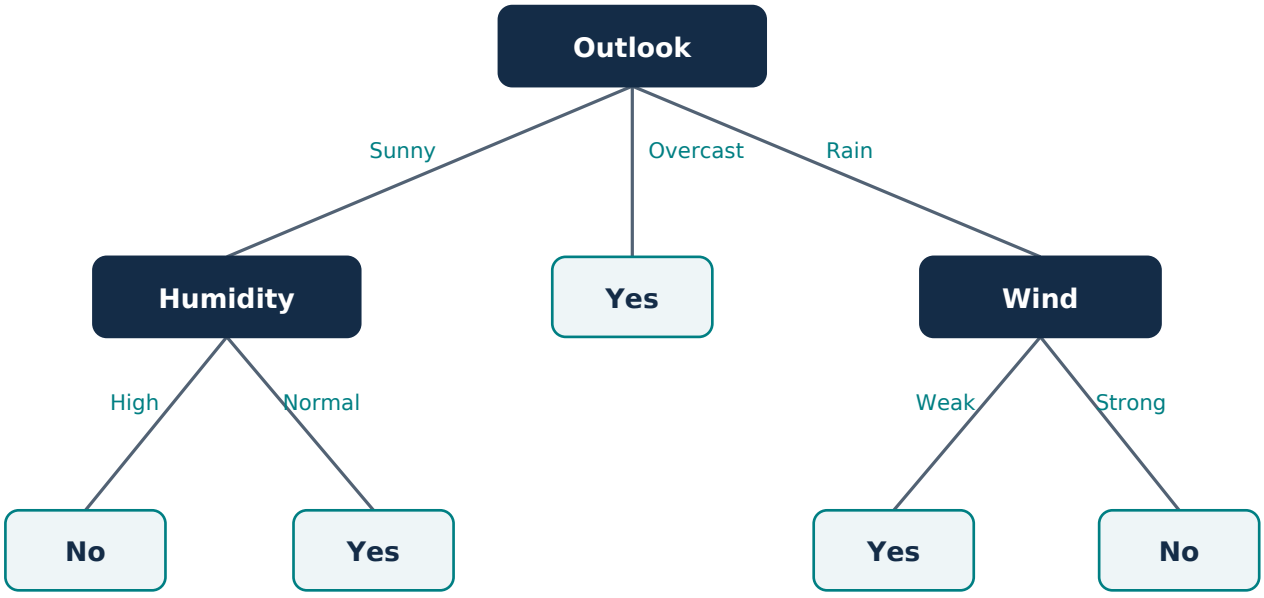
# 3. Resultados de los archivos

Los cuatro archivos de data.zip se conservaron sin modificar sus bytes. La clase objetivo es la última columna. data.csv y Play.csv son idénticos; se ejecutaron ambos y también los otros dos conjuntos distintos.

| Archivo | Filas | Raíz | H inicial | IG raíz | Aciertos |
|---|---|---|---|---|---|
| data.csv | 14 | Outlook | 0.940286 | 0.246750 | 14/14 |
| Play.csv | 14 | Outlook | 0.940286 | 0.246750 | 14/14 |
| play7.csv | 12 | Patrons | 1.000000 | 0.540852 | 12/12 |
| play8.csv | 14 | Income | 1.530619 | 0.966324 | 14/14 |

Los aciertos comparan las predicciones con los registros de entrenamiento. Validan el ajuste de los árboles a esos datos; no constituyen una estimación de rendimiento ante datos desconocidos.

## Árbol obtenido para data.csv y Play.csv




Ejemplo de lectura: si Outlook es Sunny, se consulta Humidity. High conduce a No y Normal a Yes. Si Outlook es Overcast, el árbol predice Yes sin realizar otra pregunta.

Las salidas completas, incluidos los cálculos en todos los nodos y las predicciones de cada registro, se encuentran en docs/evidencias/proporcionados_*.txt.

# 4. Otros árboles de prueba

## Restaurante: play7.csv

Contiene 12 registros y 10 atributos. La clase es Wait, con seis Yes y seis No. Patrons es la raíz: su entropía condicional es 0.459148 y su ganancia es 0.540852 bits. Los símbolos £ y el encabezado original Wait Esttimate se preservan.

```
[Patrons]
  Some -> Clase = Yes
  Full -> [Hungry]
    Yes -> [Type]
      Thai -> [Fri/Sat]
        No -> Clase = No
        Yes -> Clase = Yes
      Italian -> Clase = No
      Burger -> Clase = Yes
    No -> Clase = No
  None -> Clase = No
```

## Riesgo crediticio: play8.csv

Contiene 14 registros y cuatro atributos; la clase risk tiene seis high, tres moderate y cinco low. Income obtiene ganancia 0.966324, por lo que es la raíz. Los intervalos de ingresos se interpretan como categorías y se conserva el encabezado original Colleteral.

```
[Income]
  $0 to $15k -> Clase = high
  $15 to $35k -> [Credit History]
    unknown -> [Debt]
      high -> Clase = high
      low -> Clase = moderate
    good -> Clase = moderate
    bad -> Clase = high
  over $35k -> [Credit History]
    unknown -> Clase = low
    bad -> Clase = moderate
    good -> Clase = low
```

Estos resultados demuestran lectura de encabezados diferentes, cantidades distintas de atributos y clasificación con más de dos clases.

# 5. Ejecución y formato de entrada

## Demostración en Eclipse

1. Utilizar Eclipse IDE for Java Developers con Java 17 o superior. Descomprimir la entrega.

2. Abrir File > Import > General > Existing Projects into Workspace y seleccionar la carpeta que contiene .project.

3. Abrir src/id3/Main.java y elegir Run As > Java Application. La ejecución predeterminada carga data/proporcionados/data.csv.

4. Para otro archivo, abrir Run > Run Configurations > Java Application > Main > Arguments y escribir su ruta en Program arguments.

5. Ejecutar test/id3/Id3Test.java como Java Application para comprobar las 39 pruebas.

```
data/proporcionados/play7.csv
data/proporcionados/play8.csv risk
```

Las líneas anteriores son dos ejemplos independientes de argumentos. La segunda especifica la columna objetivo por nombre. El directorio de trabajo debe ser la raíz del proyecto, valor predeterminado en Eclipse para esta ejecución.

## Terminal y aplicación compilada

```
java -Dfile.encoding=UTF-8 -jar ID3.jar
java -Dfile.encoding=UTF-8 -jar ID3.jar data/proporcionados/play7.csv

sh run.sh --test       # Linux/macOS con JDK 17+
run.bat --test         # Windows con JDK 17+
```

El JAR incluido se ejecuta con Java 17 o superior. Para recompilar se necesita un JDK; los scripts recompilan el código antes de ejecutar. Eclipse también compila directamente las fuentes. No se necesitan bibliotecas externas, servicios ni conexión a Internet para ejecutar el proyecto.

## Reglas de los CSV

Encabezados únicos, valores UTF-8 y la misma cantidad de columnas en cada fila. Se aceptan coma o punto y coma, comillas CSV y líneas vacías. Se recortan los espacios exteriores de los campos. No se aceptan campos vacíos ni ?. La última columna es el objetivo por defecto.

La implementación trata todos los atributos como categorías, incluso números e intervalos. No realiza discretización continua, imputación ni poda. Los valores desconocidos durante predict usan la mayoría del nodo alcanzado. Los errores de entrada producen mensajes claros y un código de salida diferente de cero.

# 6. Validación y entrega

## Evidencia técnica

La compilación se verificó con Eclipse Compiler for Java 3.39.0, destino Java 17, y la ejecución con OpenJDK 21. Se completaron 39 pruebas con cero fallos, además de las ejecuciones de los ocho CSV. Los resultados de consola se guardaron en docs/evidencias/. La demostración visual dentro de Eclipse debe realizarse al grabar el video.

| Verificación | Resultado |
|---|---|
| Cuatro archivos originales | 14/14, 14/14, 12/12 y 14/14 aciertos de entrenamiento. |
| Cálculos conocidos y árbol de tenis | Entropía, cuatro ganancias y estructura completa comprobadas. |
| XOR y clasificación multiclase | XOR: 4/4; frutas: 6/6. play8.csv también comprueba tres clases. |
| Datos contradictorios | 2/3 esperado: la misma entrada tiene clases diferentes; se elige mayoría. |
| CSV y casos límite | BOM, CRLF, comillas, separadores, errores, clases puras y valores nuevos. |

## Correspondencia con la lista de cotejo

| Requisito de la guía | Material o acción |
|---|---|
| Portada y tabla de clases | Páginas 1 y 2 del informe. |
| IDE, estructura y ejecución | Proyecto importable en Eclipse y pasos en la página 6. Grabar la evidencia. |
| Resultado correcto y entropías | Código Java, cálculos, árboles y salidas reales de consola. |
| Otros archivos de prueba | Ejecuciones de los cuatro CSV originales y ejemplos adicionales. |
| Entrega puntual y video | Grabar según docs/guion_video.md y entregar en Brightspace en la fecha indicada. |

La guía asigna seis puntos a las tres funcionalidades principales, dos a cada una. El video debe mostrar el IDE, la estructura y las ejecuciones. El documento y las salidas incluidas no sustituyen esa grabación. La fecha concreta debe consultarse en Brightspace; el PDF solo indica las 23:59 del día señalado allí.

## Conclusión y referencias

El proyecto construye árboles ID3 a partir de datos categóricos, hace visible el cálculo que determina cada división y procesa correctamente los archivos suministrados. Las pruebas también verifican comportamientos que suelen fallar: empate, ganancia inicial cero, clases múltiples y entradas inválidas.

Material de la actividad: “Guía para la entrega de actividades — Proyecto ID3, Actividad 2”, PDF proporcionado; archivos de prueba data.zip proporcionados. Referencia del algoritmo: Quinlan, J. R. (1986), “Induction of Decision Trees”, Machine Learning, 1, 81–106. DOI: 10.1007/BF00116251.